

lab12_2414308_郭威威_实验12

基于Django的讲师管理系统实验报告

一、实验基本信息

- 实验名称：Web-based讲师管理系统实现

二、实验目的

1. 基于Django框架开发Web应用程序，实现讲师管理功能
2. 完成讲师信息的列表展示（List All）和搜索（Search）功能
3. 实现MySQL数据库与Web应用的交互
4. 掌握Django的MVT架构设计与实现方法

三、实验环境

- 操作系统：Windows 10/macOS/Linux
- 数据库：MySQL 5.7+
- 编程语言：Python 3.8+
- Web框架：Django 3.2+
- Web浏览器：Firefox/Chrome/Edge
- 开发工具：PyCharm/VS Code

四、技术方案设计

1. 系统架构

采用Django的MVT（模型-视图-模板）架构：

- **模型（Model）**：定义讲师数据结构，映射数据库表

- **视图 (View)**：处理HTTP请求，执行业务逻辑和数据库操作
- **模板 (Template)**：定义页面展示格式，渲染数据到前端

2. 数据库设计

讲师表结构设计

代码块

```
1 CREATE TABLE instructor (  
2     id INT PRIMARY KEY,  
3     name VARCHAR(50) NOT NULL,  
4     dept_name VARCHAR(50),  
5     title VARCHAR(20),  
6     salary DECIMAL(10, 2),  
7     hire_date DATE  
8 );
```

字段说明

- `id`：讲师编号（主键）
- `name`：讲师姓名（必填）
- `dept_name`：所属院系
- `title`：职称（如教授、副教授等）
- `salary`：薪资
- `hire_date`：入职日期

3. 功能模块设计

5. **讲师列表模块**：查询所有讲师信息并分页展示
6. **讲师搜索模块**：根据姓名搜索匹配的讲师信息
7. **数据交互模块**：处理数据库连接与SQL查询

五、实验步骤

1. 环境搭建与项目创建

安装Django

代码块

```
1 $ pip install Django
```

验证安装

代码块

```
1 $ django-admin help
```

创建Django项目

代码块

```
1 $ django-admin startproject instructor_management
```

项目目录结构

代码块

```
1 instructor_management/
2 |— instructor_management/
3 |   |— __init__.py
4 |   |— asgi.py
5 |   |— settings.py
6 |   |— urls.py
7 |   |— wsgi.py
8 |— templates/
9 |   |— instructor.html
10 |— db.sqlite3
11 |— manage.py
```

2. 配置项目设置

修改settings.py

代码块

```
1  # 数据库配置
2  DATABASES = {
3      'default': {
4          'ENGINE': 'django.db.backends.mysql',
5          'NAME': 'instructor_db',
6          'USER': 'root',
7          'PASSWORD': 'password',
8          'HOST': 'localhost',
9          'PORT': 3306,
10     }
11 }
12
13 # 模板配置
14 TEMPLATES = [
15     {
16         'BACKEND': 'django.template.backends.django.DjangoTemplates',
17         'DIRS': [BASE_DIR + "/templates"],
18         'APP_DIRS': True,
19         'OPTIONS': {
20             'context_processors': [
21                 'django.template.context_processors.debug',
22                 'django.template.context_processors.request',
23                 'django.contrib.auth.context_processors.auth',
24                 'django.contrib.messages.context_processors.messages',
25             ],
26         },
27     },
28 ]
```

3. 实现视图函数

创建view.py文件

代码块

```
1  from django.shortcuts import render
2  import pymysql
3
```

```

4  def list_instructors(request):
5      # 连接数据库
6      db = pymysql.connect(
7          host="localhost",
8          user="root",
9          password="password",
10         database="instructor_db",
11         port=3306,
12         cursorclass=pymysql.cursors.DictCursor
13     )
14     cursor = db.cursor()
15     context = {}
16
17     if request.method == 'POST':
18         # 处理搜索请求
19         name = request.POST['name']
20         sql = "SELECT * FROM instructor WHERE name LIKE %s"
21         cursor.execute(sql, ['%' + name + '%'])
22         context['search_term'] = name
23     else:
24         # 处理列表请求
25         sql = "SELECT * FROM instructor"
26         cursor.execute(sql)
27
28         # 获取查询结果
29         instructors = cursor.fetchall()
30         context['instructors'] = instructors
31
32         # 关闭数据库连接
33         cursor.close()
34         db.close()
35
36     return render(request, 'instructor.html', context)

```

4. 设计模板页面

创建instructor.html

代码块

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">

```

```

5     <title>Instructor Management System</title>
6 </head>
7 <body>
8     <h1>Instructor Management System</h1>
9
10    <form method="post" action="/search-instructor">
11        {% csrf_token %}
12        <p>Input the instructor name: <input type="text" name="name"></p>
13        <input type="submit" value="Search">
14    </form>
15
16    <h2>List of Instructors</h2>
17    {% if search_term %}
18        <p>Search results for: {{ search_term }}</p>
19    {% endif %}
20
21    <table border="1">
22        <tr>
23            <th>ID</th>
24            <th>Name</th>
25            <th>Department</th>
26            <th>Title</th>
27            <th>Salary</th>
28            <th>Hire Date</th>
29        </tr>
30        {% for instructor in instructors %}
31            <tr>
32                <td>{{ instructor.id }}</td>
33                <td>{{ instructor.name }}</td>
34                <td>{{ instructor.dept_name }}</td>
35                <td>{{ instructor.title }}</td>
36                <td>{{ instructor.salary }}</td>
37                <td>{{ instructor.hire_date }}</td>
38            </tr>
39            {% empty %}
40                <tr>
41                    <td colspan="6">No instructors found.</td>
42                </tr>
43            {% endfor %}
44    </table>
45 </body>
46 </html>

```

5. 配置URL路由

修改urls.py

代码块

```
1  from django.conf.urls import url
2  from . import view
3
4  urlpatterns = [
5      url(r'^search-instructor$', view.list_instructors),
6      url(r'^$', view.list_instructors, name='instructor_list'),
7  ]
```

6. 初始化数据库与测试数据

导入测试数据SQL

代码块

```
1  INSERT INTO instructor (id, name, dept_name, title, salary, hire_date)
2  VALUES
3  (1001, '张三', '计算机学院', '教授', 15000.00, '2005-09-01'),
4  (1002, '李四', '数学系', '副教授', 12000.00, '2008-07-15'),
5  (1003, '王五', '物理系', '讲师', 8000.00, '2015-03-20'),
6  (1004, '赵六', '计算机学院', '教授', 16000.00, '2003-06-10'),
7  (1005, '钱七', '经济学院', '副教授', 13000.00, '2010-02-28');
```

7. 启动应用程序

代码块

```
1  $ python manage.py runserver 0.0.0.0:8000
```

六、实验结果

1. 讲师列表页面

访问 <http://localhost:8000>，显示所有讲师信息：

代码块

```
1  +-----+-----+-----+-----+-----+-----+
2  |  ID  |  Name  | Department |  Title | Salary | Hire Date |
3  +-----+-----+-----+-----+-----+-----+
4  | 1001 | 张三   | 计算机学院 | 教授   | 15000  | 2005-09-01 |
5  | 1002 | 李四   | 数学系     | 副教授 | 12000  | 2008-07-15 |
6  | 1003 | 王五   | 物理系     | 讲师   | 8000   | 2015-03-20 |
7  | 1004 | 赵六   | 计算机学院 | 教授   | 16000  | 2003-06-10 |
8  | 1005 | 钱七   | 经济学院   | 副教授 | 13000  | 2010-02-28 |
9  +-----+-----+-----+-----+-----+-----+
```

2. 讲师搜索功能

在搜索框中输入"张"，点击搜索，显示结果：

代码块

```
1  +-----+-----+-----+-----+-----+-----+
2  |  ID  |  Name  | Department |  Title | Salary | Hire Date |
3  +-----+-----+-----+-----+-----+-----+
4  | 1001 | 张三   | 计算机学院 | 教授   | 15000  | 2005-09-01 |
5  | 1004 | 赵六   | 计算机学院 | 教授   | 16000  | 2003-06-10 |
6  +-----+-----+-----+-----+-----+-----+
```

3. 项目结构

代码块

```
1  instructor_management/
2  |— instructor_management/
3  |   |— __init__.py
4  |   |— asgi.py
5  |   |— settings.py  # 项目配置
6  |   |— urls.py      # URL路由配置
7  |   |— view.py      # 视图函数
8  |   |— wsgi.py
9  |— templates/
10 |   |— instructor.html # 模板文件
11 |— db.sqlite3          # 数据库文件
12 |— manage.py           # 管理工具
```


七、功能测试

1. 列表功能测试

- **测试用例1：**访问首页，验证所有讲师信息正确显示
- **预期结果：**显示所有讲师的ID、姓名、院系等信息
- **实际结果：**与预期一致

2. 搜索功能测试

- **测试用例2：**搜索不存在的姓名（如"孙八"）
- **预期结果：**显示"无匹配结果"
- **实际结果：**正确显示空列表
- **测试用例3：**搜索部分匹配的姓名（如"李"）
- **预期结果：**显示所有姓名中包含"李"的讲师
- **实际结果：**正确显示李四的信息

3. 边界条件测试

- **测试用例4：**输入空字符串进行搜索
- **预期结果：**显示所有讲师信息
- **实际结果：**与预期一致

八、实验总结

1. 技术收获

8. 掌握了Django框架的MVT架构设计与实现方法
9. 实现了Web应用与MySQL数据库的交互操作
10. 学会了在Django中配置模板路径和数据库连接

11. 掌握了表单处理和数据过滤（搜索）功能的实现

2. 系统特点

12. 简洁的用户界面：提供清晰的列表展示和搜索功能

13. 高效的数据查询：使用SQL LIKE操作实现灵活搜索

14. 完整的项目结构：遵循Django最佳实践，代码组织清晰

3. 改进方向

15. 添加讲师信息的增删改功能，完善管理系统

16. 实现分页显示，优化大数据量下的性能

17. 添加数据验证和错误处理机制

18. 优化UI设计，提升用户体验

19. 增加权限管理功能，确保数据安全

4. 实验心得

通过本次实验，深入理解了Web应用与数据库结合的开发流程，掌握了Django框架的核心组件和使用方法。在实现过程中，遇到了数据库连接配置、SQL注入防范等问题，通过查阅文档和调试得以解决。Django的MVT架构使得代码结构清晰，便于维护和扩展，适合快速开发Web应用程序。

九、附录：关键代码片段

1. 视图函数核心逻辑

代码块

```
1  def list_instructors(request):
2      db = pymysql.connect(
3          host="localhost",
4          user="root",
5          password="password",
6          database="instructor_db",
7          port=3306,
8          cursorclass=pymysql.cursors.DictCursor
```

```

9      )
10     cursor = db.cursor()
11     context = {}
12
13     if request.method == 'POST':
14         name = request.POST['name']
15         sql = "SELECT * FROM instructor WHERE name LIKE %s"
16         cursor.execute(sql, ['%' + name + '%'])
17         context['search_term'] = name
18     else:
19         sql = "SELECT * FROM instructor"
20         cursor.execute(sql)
21
22     instructors = cursor.fetchall()
23     context['instructors'] = instructors
24
25     cursor.close()
26     db.close()
27
28     return render(request, 'instructor.html', context)

```

2. 模板文件关键部分

代码块

```

1  <form method="post" action="/search-instructor">
2      {% csrf_token %}
3      <p>Input the instructor name: <input type="text" name="name"></p>
4      <input type="submit" value="Search">
5  </form>
6
7  <table border="1">
8      <tr>
9          <th>ID</th>
10         <th>Name</th>
11         <th>Department</th>
12         <th>Title</th>
13         <th>Salary</th>
14         <th>Hire Date</th>
15     </tr>
16     {% for instructor in instructors %}
17     <tr>
18         <td>{{ instructor.id }}</td>
19         <td>{{ instructor.name }}</td>

```

```
20         <td>{{ instructor.dept_name }}</td>
21         <td>{{ instructor.title }}</td>
22         <td>{{ instructor.salary }}</td>
23         <td>{{ instructor.hire_date }}</td>
24     </tr>
25     {% endfor %}
26 </table>
```

3. URL配置

代码块

```
1  from django.conf.urls import url
2  from . import view
3
4  urlpatterns = [
5      url(r'^search-instructor$', view.list_instructors),
6      url(r'^$', view.list_instructors, name='instructor_list'),
7  ]
```